



University of Camerino

SCHOOL OF SCIENCES AND TECHNOLOGIES

Master Degree in Computer Science (Class LM-18)

Knowledge Engineering and Business Technology: Personalized Menu

Group Members

Leonardo Salpini - 131055

Mohit Vijay Saini - 131054

Supervisors

Prof. Dr. Knut Hinklemann

Prof. Dr. Emanuele Laurenzi

A.A. 2024/2025

Contents

1	Introduction	9
1.1	Project Description	9
1.2	Project Objectives	9
2	Knowledge-Based Solutions	11
2.1	Decision Tables	11
2.1.1	Overview	11
2.1.2	User preferences input	11
2.1.3	Suggested dishes by diet	13
2.1.4	Suggested dishes by allergies	13
2.1.5	Dish ingredients business knowledge	13
2.1.6	Final suggestion	13
2.1.7	Executions	15
2.2	Prolog	17
2.2.1	Facts	17
2.2.2	Rules	18
2.2.3	Queries	21
2.3	Knowledge Graph/Ontology	23
2.3.1	Prefixes	23
2.3.2	Class Definitions	23
2.3.3	Property Definitions	25
2.3.4	SWRL Rules	25
2.3.5	SPARQL Queries	26
2.3.6	SHACL Shapes	31
3	Agile and Ontology-based Meta-modelling	35
3.1	BPMN 2.0 Model	35
3.2	Jena Fuseki Queries	37
3.2.1	Prefixes	37
3.2.2	First query	37
3.2.3	Second query	38
4	Conclusions	41
4.1	Leonardo Salpini	41

4.2 Mohit Vijay Saini	41
---------------------------------	----

List of Codes

2.1	Prolog ingredients facts	17
2.2	Prolog dishes facts	17
2.3	Prolog clients facts	18
2.4	Ingredient classification rules	19
2.5	Vegan dish classification rules	19
2.6	Vegetarian dish classification rules	19
2.7	Omnivore dish classification rules	20
2.8	Allergens dish classification rules	20
2.9	Calories dish classification rules	21
2.10	Final	21
2.11	Prefixes	23
2.12	SWRL Rule 1	26
2.13	SWRL Rule 2	26
2.14	SPARQL Query 1	26
2.15	SPARQL Query 2	27
2.16	SPARQL Query 3	28
2.17	SPARQL Query 4	28
2.18	SPARQL Query 5	29
2.19	SPARQL Query 6	30
2.20	SPARQL Query 1 using SWRL Rules	30
2.21	Guest Shape	31
2.22	Ingredient Shape	32
2.23	Allergen Shape	33
2.24	Diet Shape	33
2.25	Dish Shape	33
3.1	Prefixes used in Jena Fuseki	37
3.2	First query	37
3.3	Second query	39

List of Figures

2.1	Decision tables overview	12
2.2	User preferences input	12
2.3	Suggested by diet decision table	13
2.4	Final suggestion table	14
2.5	Dish ingredients business knowledge	14
2.6	Final suggestion table	15
2.7	Execution for an omnivore client, featuring a selection of gluten-free dishes and no caloric restrictions.	15
2.8	Execution for an vegetarian client, featuring a selection of gluten-free and eggs-free dishes and caloric restrictions.	16
2.9	Execution for an vegan client, featuring a selection of gluten-free dishes and caloric restrictions.	16
2.10	Prolog queries example n.1	22
2.11	Prolog queries example n.2	22
2.12	Prolog queries example n.3	22
2.13	Prolog queries example n.4	22
2.14	Class Definitions.	24
2.15	Property Definitions.	25
2.16	SWRL rules results.	26
2.17	SPARQL Query 1 results.	27
2.18	SPARQL Query 2 results.	28
2.19	SPARQL Query 3 results.	29
2.20	SPARQL Query 4 results.	29
2.21	SPARQL Query 5 results.	30
2.22	SPARQL Query 6 results.	30
3.1	BPMN 2.0 Model using AOAME	36
3.2	Custom tasks properties	36
3.3	Jena Fuseki query 1 using dynamic data from BPMN 2.0.	38
3.4	Jena Fuseki query 2 using dynamic data from BPMN 2.0.	40

1. Introduction

1.1 Project Description

Many restaurants have their menus digitized. Guests can scan a QR code and have the menu presented on their smartphones. A disadvantage is that the screen is very small and it is difficult to get an overview, in particular, if the menu is large. However, some guests can not or do not want every meal, e.g. vegetarians or guests with an allergy. Instead of showing all the meals that are offered, it would be preferable to show only those meals the guest prefers. The objective of the project is to represent the knowledge about meals and guest preferences and create a system that allows to select those meals that fit the guest preferences. The knowledge base shall contain information about typical meals of an Italian restaurant, e.g. pizza, pasta, and main dishes. Meals consist of ingredients. There are different types of ingredients like meat, vegetables, fruits, or dairy. For each ingredient, there is information about the calories. Guests can be carnivores, vegetarians, calorie-conscious, or suffer from allergies, e.g. lactose or gluten intolerance.

1.2 Project Objectives

1. Create different knowledge-based solutions for recommending food depending on the profile of a guest (carnivores, vegetarians, calorie-conscious, suffering from allergies etc.) using the following representation languages:
 - Decision tables (including DRD with sub-decisions and corresponding decision tables);
 - Prolog (including facts and rules);
 - Knowledge graph/Ontology (including rules in SWRL, queries in SPARQL and SHACL shapes)
2. Agile and ontology-based meta-modelling: adapt BPMN 2.0 to suggest the meals for a given customer. For this, you can re-use or extend the knowledge graph/ontology created in the previous task. One option that you have is to specify the class BPMN task with a new class and add additional properties, similar to what we have done in class with the Business Process as a Service case. Think of a new graphical notation for the new modelling element, which could be easy to understand for the restaurant manager. Use the triple store interface (Jena Fuseki) to fire the query result.

All the project files can be found in the [Github](#) Repository

2. Knowledge-Based Solutions

In this chapter, we show how we build our knowledge-based solutions. We start with decision tables, move to Prolog for rules, and finish with ontologies to connect information. To clarify, we made two choices: we use the word omnivore instead of carnivore to represent people who can eat any type of food, and for people counting calories, we use a `max_calories` setting for their dish limit. Lastly, we assumed that carnivores can eat vegetarian and vegan food, and that vegetarians can eat vegan food.

2.1 Decision Tables

In this section we are going to describe our approach using the decision tables, initially we make an general overview of how we implemented it and then go in depth for each element.

2.1.1 Overview

Our decision-making model in Figure 2.1 generates dish recommendations by first processing a user's dietary and allergy information through two parallel sub-decisions. The *SuggestedByDiet* table generates a list of allowed ingredients based on the diet. At the same time, the *SuggestedByAllergies* table creates a list of safe ingredients by filtering out specified allergens. The final *Suggested Dishes* takes as input the two previous lists and the number of maximum calories that the user wants to eat. By using the supporting *DishIngredients* business knowledge the last table is able to find all the dishes that the user can eat.

2.1.2 User preferences input

As show in Figure 2.2, We defined a custom data type that aggregates three distinct inputs to identify user preferences.

1. **Diet:** An enumeration that restricts the input to one of the currently supported dietary types: omnivore, vegetarian, or vegan.
2. **Allergies:** A custom data structure composed of four boolean values, each specifying an allergen to be excluded from the dish.
3. **MaxCalories:** A simple numerical value that defines the maximum permissible calorie count for any suggested dish.

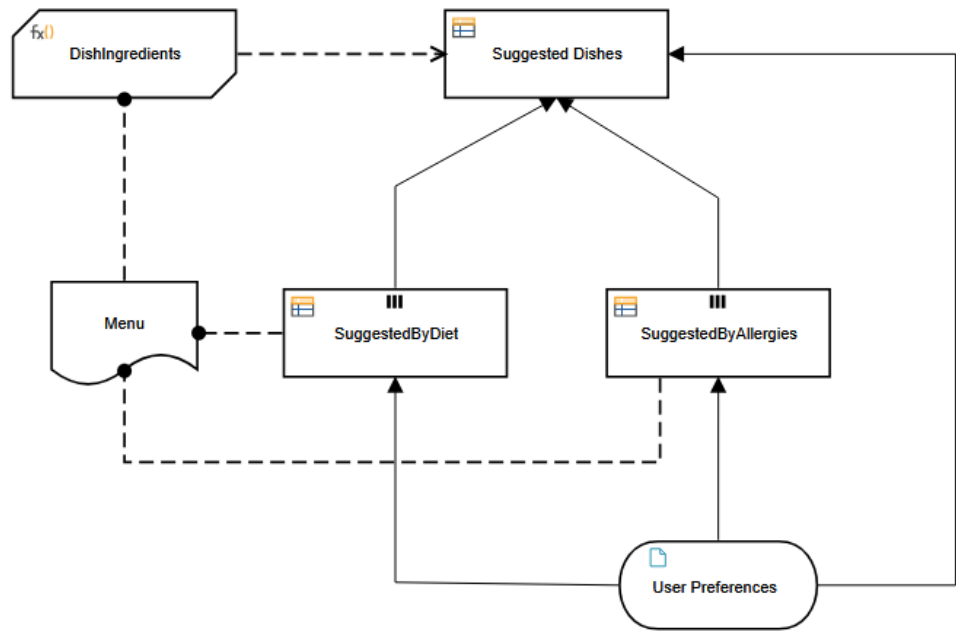


Figure 2.1: Decision tables overview

tUserPreferences	1	Diet	tDiet Text "omnivore", "vegetarian", "vegan"		
	2	Allergies tAllergies	1	GlutenFree	Boolean
			2	FishFree	Boolean
			3	MilkFree	Boolean
			4	EggsFree	Boolean
	3	MaxCalories	Number		

Figure 2.2: User preferences input

2.1.3 Suggested dishes by diet

As shown in Figure 2.3, the first subdecision takes the diet type as input and returns a custom type of ingredients that is a simple list of enumerated ingredients.

	inputs	outputs	annotations
C	User Preferences.Diet	SuggestedByDiet	Description
	tDiet "omnivore", "vegetarian", "vegan"	tIngredients "rice", "saffron", "onion", "butter", "parmesan cheese", "wheat pasta", "eggs", "pork cheek", "pecorino cheese", "black pepper", "ground beef", "tomato sauce", "bechamel sauce", "clams", "garlic", "olive oil", "parsley", "white wine", "beef steak", "salt", "chicken", "wheat flour", "lemon", "lady fingers", "mascarpone cheese", "sugar", "coffee", "cocoa powder", "water", "ripe tomatoes", "basil", "bell pepper", "zucchini", "eggplant", "salmon filet"	
	1 "omnivore", "vegetarian", "vegan"	"rice"	
	2 "omnivore", "vegetarian", "vegan"	"saffron"	
	3 "omnivore", "vegetarian", "vegan"	"onion"	
	4 "omnivore", "vegetarian"	"butter"	
	5 "omnivore", "vegetarian"	"parmesan cheese"	

Figure 2.3: Suggested by diet decision table

2.1.4 Suggested dishes by allergies

This decision shown in Figure 2.4 table generates a list of suitable dishes based on a user's allergy preferences. The inputs are boolean choices for diets free of gluten, fish, milk, or eggs. The table operates on a set of rules where an ingredient is suggested only if it does not conflict with the user's choice. For example, "wheat pasta" is added to the list only if the user's GlutenFree preference is false. Ingredients like "rice" and "saffron" are always included, as the dashes (-) indicate they are considered safe regardless of these specific allergy inputs. The final output is a list of all ingredients that pass these conditions.

2.1.5 Dish ingredients business knowledge

The Business Knowledge Model in Figure 2.5 takes a dish name and lists its ingredients. Although using this model was not a necessity, we found it helpful because it made the final decision table much clearer and easier to read.

2.1.6 Final suggestion

The objective is to generate a final list of suitable dishes, the table in Figure 2.6 takes as inputs a list of approved ingredients from the SuggestedByAllergies table, a second list of approved ingredients from SuggestedByAllergies table, and the maximum number of

C	inputs				outputs	annotations
	User Preferences.Allergies.GlutenFree	User Preferences.Allergies.FishFree	User Preferences.Allergies.MilkFree	User Preferences.Allergies.EggsFree	SuggestedByAllergies	Description
	Boolean	Boolean	Boolean	Boolean	<i>Ingredients</i> "rice", "saffron", "onion", "butter", "parmesan cheese", "wheat pasta", "eggs", "pork cheek", "pecorino cheese", "black pepper", "ground beef", "tomato sauce", "bechamel sauce", "clams", "garlic", "olive oil", "parsley", "white wine", "beef steak", "salt", "chicken", "wheat flour", "lemon", "lady fingers", "mascarpone cheese", "sugar", "coffee", "cocoa powder", "water", "ripe tomatoes", "basil", "bell pepper", "zucchini", "eggplant", "salmon filet"	
1	-	-	false	-	"parmesan cheese"	
2	-	-	false	-	"butter"	
3	false	-	-	-	"wheat pasta"	
4	-	-	-	false	"eggs"	
5	-	-	-	-	"rice"	

Figure 2.4: Final suggestion table

C	inputs	outputs	annotations
	User Preferences.Diet	SuggestedByDiet	Description
	<i>tDiet</i> "omnivore", "vegetarian", "vegan"	<i>Ingredients</i> "rice", "saffron", "onion", "butter", "parmesan cheese", "wheat pasta", "eggs", "pork cheek", "pecorino cheese", "black pepper", "ground beef", "tomato sauce", "bechamel sauce", "clams", "garlic", "olive oil", "parsley", "white wine", "beef steak", "salt", "chicken", "wheat flour", "lemon", "lady fingers", "mascarpone cheese", "sugar", "coffee", "cocoa powder", "water", "ripe tomatoes", "basil", "bell pepper", "zucchini", "eggplant", "salmon filet"	
1	"omnivore", "vegetarian", "vegan"	"rice"	
2	"omnivore", "vegetarian", "vegan"	"saffron"	
3	"omnivore", "vegetarian", "vegan"	"onion"	
4	"omnivore", "vegetarian"	"butter"	
5	"omnivore", "vegetarian"	"parmesan cheese"	

Figure 2.5: Dish ingredients business knowledge

calories specified by the user. For each dish the system perform three checks. First, it verifies that every ingredient in the dish is contained within the allergy-safe list. Next, it performs the same validation against the second list of approved ingredients. Lastly, it checks that the dish's total calories are less than or equal to the user's specified limit. The final output is a list containing only the names of those dishes that have successfully passed all three of these checks.

	inputs			outputs	annotations
	SuggestedByDiet	SuggestedByAllergies	User Preferences.MaxCalories	Suggested Dishes	Description
C	Ingredients "rice", "saffron", "onion", "butter", "parmesan cheese", "wheat pasta", "eggs", "pork cheek", "pecorino cheese", "black pepper", "ground beef", "tomato sauce", "bechamel sauce", "clams", "garlic", "olive oil", "parsley", "white wine", "beef steak", "salt", "chicken", "wheat flour", "lemon", "bady finger", "mascarpone cheese", "sugar", "coffee", "cocoa powder", "water", "ripe tomatoes", "basil", "bell pepper", "zucchini", "eggplant", "salmon filet"	Ingredients "rice", "saffron", "onion", "butter", "parmesan cheese", "wheat pasta", "eggs", "pork cheek", "pecorino cheese", "black pepper", "ground beef", "tomato sauce", "bechamel sauce", "clams", "garlic", "olive oil", "parsley", "white wine", "beef steak", "salt", "chicken", "wheat flour", "lemon", "bady finger", "mascarpone cheese", "sugar", "coffee", "cocoa powder", "water", "ripe tomatoes", "basil", "bell pepper", "zucchini", "eggplant", "salmon filet"	Number	Any	
1	every ingredient in DishIngredients("risotto-milanese") satisfies list contains(?ingredient)	every ingredient in DishIngredients("risotto-milanese") satisfies list contains(?ingredient)	>=546	"Risotto alla Milanese"	
2	every ingredient in DishIngredients("carbonara") satisfies list contains(?ingredient)	every ingredient in DishIngredients("carbonara") satisfies list contains(?ingredient)	>=764	"Carbonara"	
3	every ingredient in DishIngredients("lasagna") satisfies list contains(?ingredient)	every ingredient in DishIngredients("lasagna") satisfies list contains(?ingredient)	>=845	"Lasagna"	
4	every ingredient in DishIngredients("spaghetti-vongole") satisfies list contains(?ingredient)	every ingredient in DishIngredients("spaghetti-vongole") satisfies list contains(?ingredient)	>=360	"Spaghetti alle Vongole"	

Figure 2.6: Final suggestion table

2.1.7 Executions

User Preferences null

Diet

omnivore

Allergies null

GlutenFree

true

FishFree

false

MilkFree

false

EggsFree

false

MaxCalories

9999

Suggested Dishes

Risotto alla Milanese

Bistecca alla fiorentina

Sorbetto al Limone

Crema Caffè

Involtni di manzo e parmigiano

Guanciale croccante con uova

Omelette al parmigiano e cipolla

Misto Verdure

Riso con pomodoro, cipolla e prezzemolo

Figure 2.7: Execution for an omnivore client, featuring a selection of gluten-free dishes and no caloric restrictions.

15

▼ User Preferences null

Diet

vegetarian

▼ Allergies null

GlutenFree

true

FishFree

false

MilkFree

false

EggsFree

true

MaxCalories

1000

Suggested Dishes

Risotto alla Milanese
Sorbetto al Limone
Crema Caffè
Misto Verdure
Riso con pomodoro, cipolla e prezzemolo

Figure 2.8: Execution for an vegetarian client, featuring a selection of gluten-free and eggs-free dishes and caloric restrictions.

▼ User Preferences null

Diet

vegan

▼ Allergies null

GlutenFree

true

FishFree

false

MilkFree

false

EggsFree

false

MaxCalories

1500

Suggested Dishes

Sorbetto al Limone
Misto Verdure
Riso con pomodoro, cipolla e prezzemolo

Figure 2.9: Execution for an vegan client, featuring a selection of gluten-free dishes and caloric restrictions.

2.2 Prolog

In this section, we will detail our Prolog implementation. We will begin by presenting the facts we have defined, then explain the rules that govern the logic, and conclude by demonstrating some example queries to show the system in action.

2.2.1 Facts

Our prolog facts are divided into three categories. The first is Ingredients, where each fact follows the structure *ingredient(Name, Origin, Calories, Allergen List)*. Talking about "origin", it refers to the source of classification of a food and the categories we used are the following: plant based, animal, animal product and mineral.

```

1 ingredient(rice, plant, 130, []).
2 ingredient(saffron, plant, 310, []).
3 ingredient(onion, plant, 40, []).
4 ingredient(butter, animal_product, 717, [milk]).
5 ingredient(parmesan_cheese, animal_product, 431, [milk]).
6 ingredient(wheat_pasta, plant, 350, [gluten]).
7 ingredient(eggs, animal_product, 143, [eggs]).
8 ingredient(pork_cheek, animal, 655, []).
9 ingredient(pecorino_cheese, animal_product, 387, [milk]).
10 ingredient(black_pepper, plant, 255, []).
11 ingredient(ground_beef, animal, 250, []).
12 ingredient(tomato_sauce, plant, 40, []).
13 ingredient(bechamel_sauce, animal_product, 105, [gluten, milk]).
14 ingredient(clams, animal, 74, [fish]).
15 ingredient(garlic, plant, 149, []).
16 ingredient(olive_oil, plant, 250, []).
17 ingredient(parsley, plant, 36, []).
18 ingredient(white_wine, plant, 82, []).
19 ingredient(beef_steak, animal, 250, []).
20 ingredient(salt, mineral, 0, []).
21 ingredient(chicken, animal, 165, []).
22 ingredient(wheat_flour, plant, 364, [gluten]).
23 ingredient(lemon, plant, 29, []).
24 ingredient(lady_fingers, animal_product, 388, [gluten, eggs]).
25 ingredient(mascarpone_cheese, animal_product, 435, [milk]).
26 ingredient(sugar, plant, 387, []).
27 ingredient(coffee, plant, 2, []).
28 ingredient(cocoa_powder, plant, 228, []).
29 ingredient(water, mineral, 0, []).
30 ingredient(ripe_tomatoes, plant, 18, []).
31 ingredient(basil, plant, 22, []).
32 ingredient(bell_pepper, plant, 17, []).
33 ingredient(zucchini, plant, 31, []).
34 ingredient(eggplant, plant, 25, []).
35 ingredient(salmon_filet, animal, 210, [fish]).

```

Code 2.1: Prolog ingredients facts

The next category of facts refers to dishes. Their structure is really simple: it consists of a name followed by a list of ingredients.

```

1 dish(sorbetto_al_limone, [lemon, water, sugar]).

```

```
2 dish(tiramisu, [lady_fingers, mascarpone_cheese, eggs, sugar, coffee
  , cocoa_powder]).
3 dish(crema_caffe, [coffee, mascarpone_cheese, water, cocoa_powder,
  sugar]).
4 dish(risotto_alla_milanese, [rice, saffron, onion, butter,
  parmesan_cheese]).
5 dish(carbonara, [wheat_pasta, eggs, pork_cheek, pecorino_cheese,
  black_pepper]).
6 dish(lasagna, [wheat_pasta, ground_beef, tomato_sauce,
  bechamel_sauce]).
7 dish(spaghetti_alle_vongole, [wheat_pasta, clams, garlic, olive_oil,
  parsley, white_wine]).
8 dish(spaghetti_al_pomodoro, [wheat_pasta, ripe_tomatoes, garlic,
  olive_oil, basil, salt]).
9 dish(pasta_al_burro_parmigiano_e_pepe_nero, [wheat_pasta, butter,
  parmesan_cheese, black_pepper, salt]).
10 dish(riso_pomodoro_cipolla, [rice, ripe_tomatoes, onion, olive_oil,
  parsley, salt, black_pepper]).
11 dish(bistecca_alla_fiorentina, [beef_steak, salt, black_pepper]).
12 dish(scaloppine_al_limone, [chicken, wheat_flour, lemon, butter,
  parsley, salt, black_pepper, olive_oil]).
13 dish(involtini_di_manzo_al_parmigiano, [beef_steak, parmesan_cheese,
  parsley, garlic, olive_oil, black_pepper, salt]).
14 dish(guanciale_croccante_con_uova, [pork_cheek, eggs,
  parmesan_cheese, black_pepper, olive_oil, salt]).
15 dish(omelette_parmigiano_cipolle, [eggs, parmesan_cheese, onion,
  butter, parsley, salt, black_pepper]).
16 dish(misto_verdure, [bell_pepper, zucchini, eggplant, garlic,
  olive_oil, parsley, salt]).
17 dish(salmone_in_crosta, [salmon_filet, wheat_flour, olive_oil,
  parsley, garlic, lemon, salt]).
```

Code 2.2: Prolog dishes facts

The final category of facts identifies the clients. They are characterized by a name, diet type, maximum calories they want to eat and a list of allergies.

```
1 client(elena, vegetarian, 1800, [gluten, milk]).
2 client(francesco, omnivore, 2200, [fish]).
3 client(sofia, vegan, 1300, []).
4 client(alessandro, omnivore, 3000, [nuts, eggs]).
5 client(giulia, vegetarian, 1400, [eggs]).
6 client(matteo, vegan, 2000, [gluten]).
7 client(chiara, omnivore, 1900, []).
8 client(davide, vegetarian, 1600, [nuts]).
9 client(beatrice, omnivore, 2800, [milk]).
10 client(riccardo, vegan, 1700, [gluten, nuts]).
```

Code 2.3: Prolog clients facts

2.2.2 Rules

This section presents the Prolog-based rule system for suggesting dishes. It describe the logic for three key constraints —diet, allergies, and calories— and concludes by describing the final predicate that combines these rules to generate a list of suitable recommendations.

Dishes Classification by Diet

The first set of rules work is used to classify ingredients. They determine whether an ingredient is animal-based, plant-based, or mineral-based. These rules form the basis of the subsequent ones. Here, we use the wildcard "_" because the only information needed to classify an ingredient is its origin. The list of allergens and calories are not useful.

```

1 is_animal_product(Ingredient) :- ingredient(Ingredient,
    animal_product, _, _).
2 is_plant(Ingredient) :- ingredient(Ingredient, plant, _, _).
3 is_mineral(Ingredient) :- ingredient(Ingredient, mineral, _, _).

```

Code 2.4: Ingredient classification rules

Following this there is the set of rules needed to determines whether a dish is vegan or not. Initially, *is_vegan_ingredient/1* classifies an ingredient as vegan if it is a plant or mineral.

Next, the *are_all_ingredients_vegan/1* predicate recursively checks a list of ingredients. It verifies that every item in the list satisfies the predicate *is_vegan_ingredient/1*. It does this by checking the first ingredient, then applying the same logic to each subsequent ingredient until the list is empty.

Finally, the main *is_vegan/1* predicate combines everything together in one rule. Take a dish as input, it gets the associated list of ingredients and calls *are_all_ingredients_vegan/1* to ensure the entire list is acceptable on a vegan diet.

At the end we added the cut "!" operator to stops the search after the first success, in this way after finding a "True" results, we block the prolog backtrack that otherwise will lead to a false.

```

1 is_vegan_ingredient(Ing) :- is_plant(Ing).
2 is_vegan_ingredient(Ing) :- is_mineral(Ing).
3
4 are_all_ingredients_vegan([]).
5 are_all_ingredients_vegan([Head|Tail]) :-
6     is_vegan_ingredient(Head),
7     are_all_ingredients_vegan(Tail).
8
9 is_vegan(Dish) :-
10     dish(Dish, Ingredients),
11     are_all_ingredients_vegan(Ingredients),
12     !.

```

Code 2.5: Vegan dish classification rules

The next group of rules is needed to indentify if a dish is vegetarian or not. Here we are going to apply the exact same concepts as before, and the code is as follow:

```

1 is_vegetarian_ingredient(Ing) :- is_plant(Ing).
2 is_vegetarian_ingredient(Ing) :- is_mineral(Ing).
3 is_vegetarian_ingredient(Ing) :- is_animal_product(Ing).
4
5 are_all_ingredients_vegetarian([]).
6 are_all_ingredients_vegetarian([Head|Tail]) :-
7     is_vegetarian_ingredient(Head),
8     are_all_ingredients_vegetarian(Tail).

```

```
9
10 is_vegetarian(Dish) :-
11     dish(Dish, Ingredients),
12     are_all_ingredients_vegetarian(Ingredients),
13     !.
```

Code 2.6: Vegetarian dish classification rules

In the end there is the rule that defines what makes a dish omnivore-friendly. It simply checks if the dish is present in our list of facts, ignoring the ingredients.

```
1 is_omnivore(Dish) :- dish(Dish, _).
```

Code 2.7: Omnivore dish classification rules

Dishes Classification by Allergens

This second section of the code identifies whether a given dish contains any allergens from a specified list. The first predicate, *ingredient_contains_allergens/2*, determines if a single ingredient contains any allergen from the user allergies list. First, it retrieves the allergens for the passed ingredient. Next, it uses two *member/2* predicates, the first member iterates through the *UserAllergies* list, selecting one Allergen at a time and foreach one the second member then checks if that same Allergen also exists in the *IngredientAllergens* list. If the predicate fails, backtracking occurs automatically leading to the first *member/2* to select the next allergen in the list and repeat the process until the list is exhausted. Since we need to find if there is one common allergen between the two list, we added the cut operator to terminate the predicate when he finds the first common allergen.

The second predicate *dish_contains_allergens/2*, determines if a dish contains any allergen from a list. It initially retrieve the dish ingredients, then by using the *member/2* predicate iterates list and for each ingredient it calls the previous described predicate. As soon as one ingredient contains one of the allergens, the predicate will terminate thanks to the cut operator.

```
1 ingredient_contains_allergens(Ingredient, AllergensList) :-
2     ingredient(Ingredient, _, _, IngredientAllergens),
3     member(Allergen, AllergensList),
4     member(Allergen, IngredientAllergens),
5     !.
6
7 dish_contains_allergens(Dish, AllergensList) :-
8     dish(Dish, IngredientsList),
9     member(Ingridient, IngredientsList),
10    ingredient_contains_allergens(Ingridient, AllergensList),
11    !.
```

Code 2.8: Allergens dish classification rules

Dishes Classification by Calories

This section of the code determines if the total calories in a dish are below the user's maximum calorie limit. The *sum_calories/1* is a recursive predicates that calculates the total calories count from a list of ingredients. It works by taking each ingredient of

the list, finding its calories and adding them to the recursively calculated sum of the rest of the list.

This predicate is used by the main one, *is_dish_below_calories/2*. This predicate takes a dish and the user max calories as input, it retrieves the list of ingredients for the dish and uses the *sum_calories/2* predicate to get total calories. In the end checks if the *TotalCalories* are lower or higher than the user limit.

```

1 sum_calories([], 0).
2 sum_calories([Ingredient | Rest], TotalCalories) :-
3     ingredient(Ingredient, _, Calories, _),
4     sum_calories(Rest, RestCalories),
5     TotalCalories is Calories + RestCalories.
6
7 is_dish_below_calories(Dish, MaxCalories) :-
8     dish(Dish, Ingredients),
9     sum_calories(Ingredients, TotalCalories),
10    TotalCalories =< MaxCalories.

```

Code 2.9: Calories dish classification rules

Final predicate for suggestion

The final predicate, *suggested_dishes/2*, generates a suggestion by using the predicate *is_dish_allowed_by_diet/2* to check the dish compatibility with the diet and combining it with all other previously described rules.

```

1 is_dish_allowed_by_diet(omnivore, Dish) :- is_omnivore(Dish).
2 is_dish_allowed_by_diet(vegetarian, Dish) :- is_vegetarian(Dish).
3 is_dish_allowed_by_diet(vegan, Dish) :- is_vegan(Dish).
4
5 suggested_dishes(ClientName, Dish) :-
6     client(ClientName, DietType, MaxCalories, Allergens),
7     dish(Dish, _),
8     allowed_dish_by_diet(DietType, Dish),
9     not(dish_contains_allergens(Dish, Allergens)),
10    is_dish_below_calories(Dish, MaxCalories).

```

Code 2.10: Final

2.2.3 Queries

The main use for the predicate *suggested_dishes(Client, Dish)* is to find all suitable dishes for a client, we can do that by providing the client's name and using a variable for the second argument, the query returns all valid dishes, as shown in Figure 2.10.

It's also possible to check a single dish by providing its name instead of a variable. The query returns true if the dish can be eaten by the client and false otherwise as shown in Figure 2.11.

The query can also be used in reverse. By providing a specific dish and using a variable for the client, the system will list all clients who can eat that particular dish as shown in Figure 2.12.

Finally, using variables for both arguments will return all possible pairs of clients and the dishes they are allowed to eat as shown in part of the output in Figure 2.13.



Figure 2.10: Prolog queries example n.1

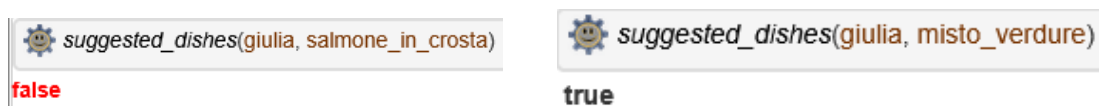


Figure 2.11: Prolog queries example n.2



Figure 2.12: Prolog queries example n.3

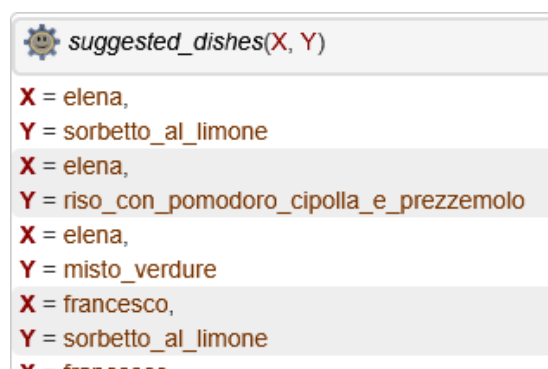


Figure 2.13: Prolog queries example n.4

2.3 Knowledge Graph/Ontology

Our knowledge graph, was developed using a combination of tools: **Protégé** and **GraphDB**. The ontology is formally represented in a `.ttl` (Turtle) file, which is a W3C standard syntax for RDF (Resource Description Framework). This format was chosen because of its ability to be both human-readable and machine-processable. The `.ttl` file is logically structured into several key sections, each defining distinct aspects of our knowledge graph's schema and data.

2.3.1 Prefixes

```

1 @prefix : <http://menu.ch#> .
2 @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
3 @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
4 @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
5 @prefix owl: <http://www.w3.org/2002/07/owl#> .

```

Code 2.11: Prefixes

This section defines the essential prefixes used to abbreviate URIs within the knowledge graph. We leverage standard prefixes for core Semantic Web vocabularies (RDF, RDFS, OWL, and XSD), along with our custom base URI `<http://menu.ch#>`, which serves as the namespace for all domain-specific entities in our ontology.

2.3.2 Class Definitions

This section defines the classes that make up the core entities of our domain, as well as their subclasses. Our class hierarchy is represented in the Figure 2.14.

Furthermore, each instance of super and subclass is characterized by object and data properties, shown in Figure 2.15.

The Guest class describes the profile of a guest. Each guest profile is made up of a list of allergies, maximum calories for a dish, the type of diet, and the name of the guest. These profiles can be used in queries to recommend dishes based on the "constraints" declared in the profile.

The Dish class describes a dish. It has 3 subclasses: MainCourse, SecondCourse, and Dessert. Each dish is composed of a name and a list of ingredients that make up the dish.

The Diet class describes the diet. We have 3 diets: Omnivore, Vegetarian, Vegan. Each diet is made up of a list of dishes that can be eaten by that type of diet.

The Allergen class describes different types of allergies. We implemented 5 allergies: Gluten, Fish, EggsAllergen, Lactose, Nuts. They all contain a single data property, name.

The Ingredient class describes all the different types of ingredients that make up the dishes on our menu. We have a lot of subclasses of ingredients, as shown in the Figure 2.14. Each ingredient is made up of name, a list of allergies, and a list of diets that can consume the dish.

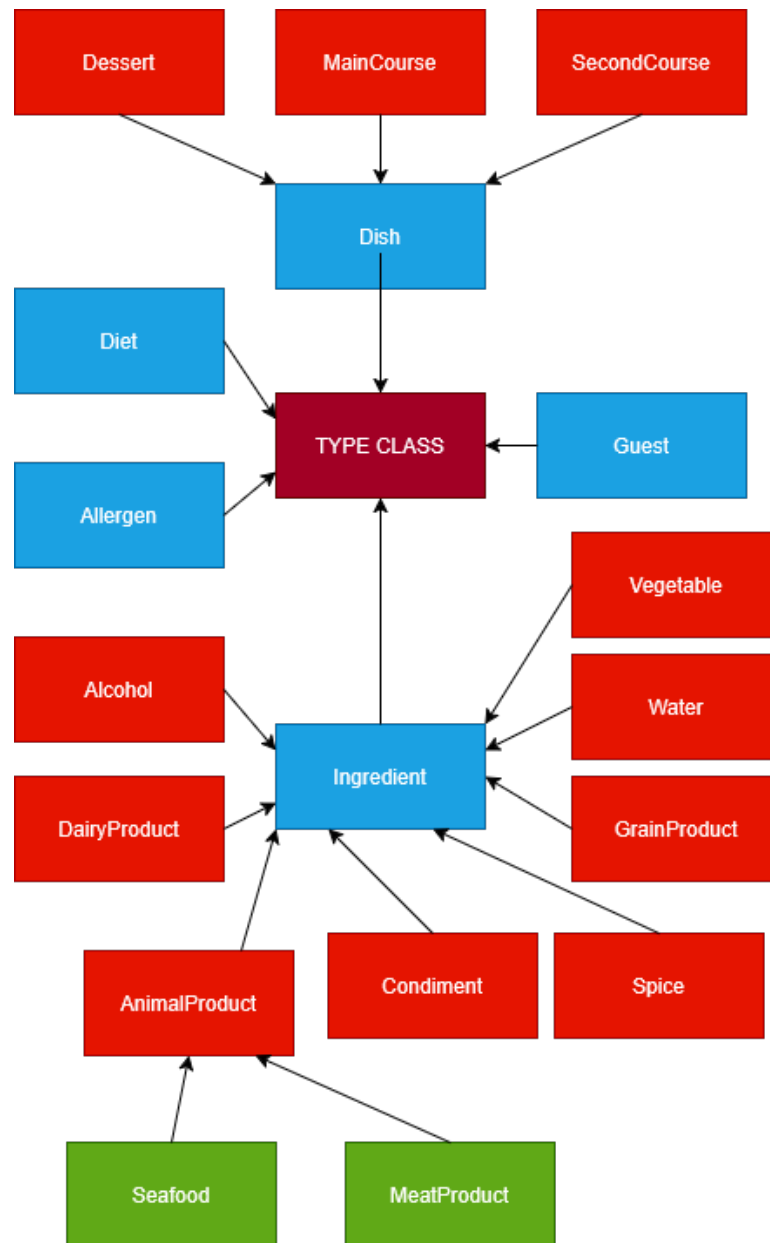


Figure 2.14: Class Definitions.

2.3.3 Property Definitions

The object property is necessary to establish relationships and to connect entities within our knowledge graph because otherwise they would just be isolated data points. So, it allows us to model complex interactions and dependencies, and it is also necessary for graph traversal because queries often involve traversing relationships, so object properties are like "edges" of a "node" allowing navigation from one entity to another. The data property is essential to attach literal values to the entities (int, string, etc.). Both properties enforce constraints using **rdfs:domain** and **rdfs:range**, which help to maintain semantic integrity and allow reasoners to detect errors. The domain keyword specifies the class of individuals that can be subject of a triple and range keyword specifies the class of individuals (for object properties) or the data type (for data properties) that can be the object of a triple.

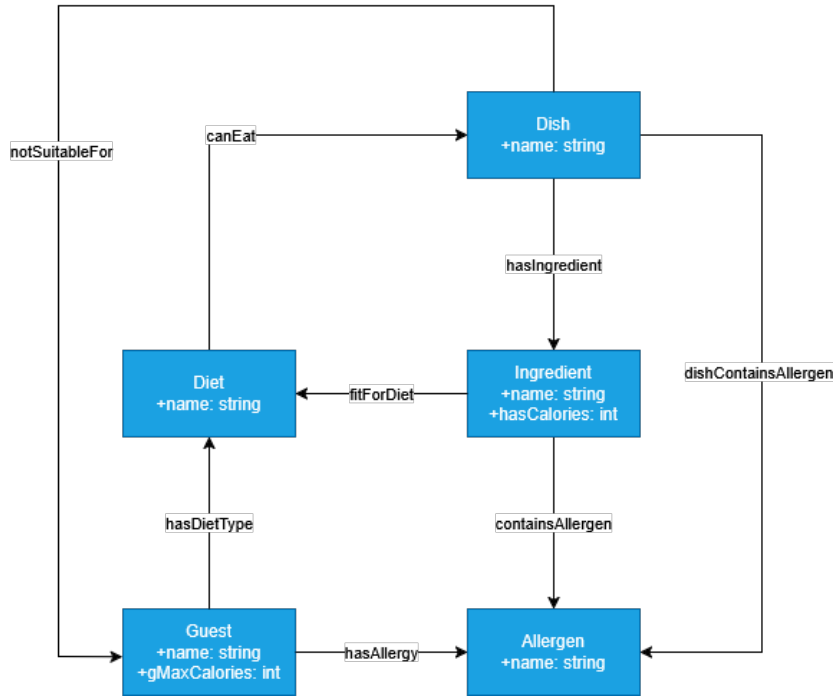


Figure 2.15: Property Definitions.

The following object properties: *dishContainsAllergen* and *notSuitableFor* are inferred using SWRL rules. This makes some queries simpler and faster. For example, they do not have to cycle through all ingredients in a dish to check if it contains guest allergy.

2.3.4 SWRL Rules

SWRL (Semantic Web Rule Language) is used to express logical rules in ontology-based systems. We used them to model how allergens in ingredients affect dish safety for guests with allergies.

The 1st rule states that if a dish ?d has an ingredient ?i, and that ingredient contains an allergen ?a, then the dish also contains that allergen. This is necessary to determine presence of the allergen at the dish level.

```
1 :Dish(?d) ^ :hasIngredient(?d, ?i) ^ :containsAllergen(?i, ?a) -> :
    dishContainsAllergen(?d, ?a)
```

Code 2.12: SWRL Rule 1

The 2nd identifies dishes that are not suitable for a guest ?g who has an allergy ?a. It checks whether any ingredient in a dish contains an allergen the guest is allergic to.

```
1 :Guest(?g) ^ :hasAllergy(?g, ?a) ^ :Dish(?d) ^ :hasIngredient(?d, ?i)
    ^ :containsAllergen(?i, ?a) -> :notSuitableFor(?d, ?g)
```

Code 2.13: SWRL Rule 2

These rules help automate allergen checks by allowing reasoning engines to infer which dishes contain allergens and which dishes should be avoided by certain guests. This allows us to write SPARQL queries that are much simpler thanks to inference reasoning.

Figure 2.16 shows the inferred rules highlighted in yellow

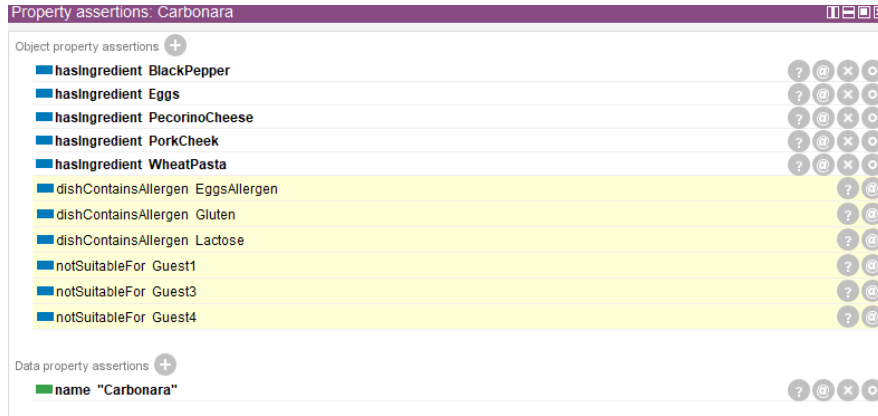


Figure 2.16: SWRL rules results.

2.3.5 SPARQL Queries

SPARQL (SPARQL Protocol and RDF Query Language) is used to query RDF data in ontologies. We used SPARQL to retrieve meals, ingredients, allergens, and their relationships to guests' diets and allergies. Some queries also leverage inferred knowledge produced by SWRL rules.

Find Meals a Guest Can Eat (Considering Diet, Allergies, and Calories)

```
1 PREFIX : <http://menu.ch#>
2
3 SELECT ?Meal (SUM(?cal) AS ?Calories)
4 WHERE {
5   VALUES ?guest { :Guest2 }
6
7   ?guest :hasDietType ?diet ;
8           :gMaxCalories ?maxCalories .
9 }
```

```

10  OPTIONAL { ?guest :hasAllergy ?guestAllergy . }
11
12  ?diet :canEat ?dish .
13  ?dish :name ?Meal ;
14         :hasIngredient ?ingredient .
15
16  ?ingredient :hasCalories ?cal .
17
18  FILTER NOT EXISTS {
19    ?guest :hasAllergy ?guestAllergy .
20    ?dish :hasIngredient ?dishIngredient .
21    ?dishIngredient :containsAllergen ?guestAllergy .
22  }
23 }
24 GROUP BY ?Meal ?maxCalories
25 HAVING (SUM(?cal) <= ?maxCalories)

```

Code 2.14: SPARQL Query 1

This query identifies all meals a specific guest (Guest2) can consume, based on their diet type, allergy restrictions, and maximum allowed calories. It first retrieves the guest's diet and calorie limit. Then it matches dishes permitted by the diet and sums up their ingredient calories. The FILTER NOT EXISTS block ensures that no ingredient in the dish contains allergens the guest is allergic to. The result is grouped by meal name and filtered by the total calorie constraint using the HAVING clause. Results are shown in Figure 2.17.

Meal	Calories
"Carbonara" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"1790" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Crema Caffè" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"1052" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Spaghetti al pomodoro" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"789" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Sorbetto al limone" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"416" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Lasagna" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"745" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Salmone in Crosta" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"1038" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Omelette al Parmigiano con Cipolle e Prezemolo" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"1622" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Tiramisù" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"1583" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Riso con Pomodoro, Cipolla e Prezemolo" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"729" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Bistecca alla Fiorentina" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"505" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Involtini di Manzo al Parmigiano" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"1371" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Guanciale Croccante con Uova" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"1734" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Scaloppine al limone" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"1816" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Risotto alla milanese" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"1628" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Misto Verdure" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"508" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Spaghetti alle vongole" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"941" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>
"Pasta al Burro, Parmigiano e Pepe Nero" ^{AA} <http://www.w3.org/2001/XMLSchema#string>	"1753" ^{AA} <http://www.w3.org/2001/XMLSchema#integer>

Figure 2.17: SPARQL Query 1 results.

Find Meals Allowed by a Diet Type

```

1  PREFIX : <http://menu.ch#>
2
3  SELECT DISTINCT ?Meal
4  WHERE {
5    VALUES ?dietType { :Vegan }
6
7    ?dietType :canEat ?dish .
8    ?dish :name ?Meal .
9

```

```

10  FILTER NOT EXISTS {
11      ?dish :hasIngredient ?ingredient .
12      FILTER NOT EXISTS {
13          ?ingredient :fitForDiet ?dietType .
14      }
15  }
16 }
17 ORDER BY ?Meal

```

Code 2.15: SPARQL Query 2

This query retrieves all distinct meal names that are permitted under a given diet type (Vegan). It ensures that all ingredients in the dish are marked as compatible with the specified diet type. The inner FILTER NOT EXISTS ensures that no disallowed ingredient is present in the dish. Results are shown in Figure 2.18.

Meal
"Misto Verdure" <http://www.w3.org/2001/XMLSchema#string>
"Riso con Pomodoro, Cipolla e Prezzemolo" <http://www.w3.org/2001/XMLSchema#string>
"Sorbetto al limone" <http://www.w3.org/2001/XMLSchema#string>
"Spaghetti al pomodoro" <http://www.w3.org/2001/XMLSchema#string>

Figure 2.18: SPARQL Query 2 results.

Find Ingredients Compatible with a Diet Type

```

1  PREFIX : <http://menu.ch#>
2
3  SELECT DISTINCT ?IngredientName
4  WHERE {
5      VALUES ?dietType { :Omnivore }
6
7      ?ingredient :fitForDiet ?dietType ;
8                  :name ?IngredientName .
9  }
10 ORDER BY ?IngredientName

```

Code 2.16: SPARQL Query 3

This query lists all ingredient names that are marked as suitable for a specific diet type (Omnivore). Part of the result shown in Figure 2.19

Find Meals Containing a Given Ingredient

```

1  PREFIX : <http://menu.ch#>
2
3  SELECT DISTINCT ?MealName
4  WHERE {
5      VALUES ?ingredient { :WheatPasta }
6
7      ?meal :hasIngredient ?ingredient ;
8            :name ?MealName .
9  }

```

```
10 ORDER BY ?MealName
```

Code 2.17: SPARQL Query 4

This query returns all meals that include a specific ingredient (WheatPasta). It is useful for identifying where a certain ingredient is used across the menu. Results are shown in Figure 2.20.

IngredientName
"Basil"^^<http://www.w3.org/2001/XMLSchema#string>
"Bechamel Sauce"^^<http://www.w3.org/2001/XMLSchema#string>
"Beef Steak"^^<http://www.w3.org/2001/XMLSchema#string>
"Bell Pepper"^^<http://www.w3.org/2001/XMLSchema#string>
"Black Pepper"^^<http://www.w3.org/2001/XMLSchema#string>
"Butter"^^<http://www.w3.org/2001/XMLSchema#string>
"Chicken"^^<http://www.w3.org/2001/XMLSchema#string>
"Clams"^^<http://www.w3.org/2001/XMLSchema#string>
"Cocoa Powder"^^<http://www.w3.org/2001/XMLSchema#string>
"Coffee"^^<http://www.w3.org/2001/XMLSchema#string>
"Egg Plant"^^<http://www.w3.org/2001/XMLSchema#string>
"Eggs"^^<http://www.w3.org/2001/XMLSchema#string>
"Garlic"^^<http://www.w3.org/2001/XMLSchema#string>

Figure 2.19: SPARQL Query 3 results.

MealName
"Carbonara"^^<http://www.w3.org/2001/XMLSchema#string>
"Lasagna"^^<http://www.w3.org/2001/XMLSchema#string>
"Pasta al Burro, Parmigiano e Pepe Nero"^^<http://www.w3.org/2001/XMLSchema#string>
"Spaghetti al pomodoro"^^<http://www.w3.org/2001/XMLSchema#string>
"Spaghetti alle vongole"^^<http://www.w3.org/2001/XMLSchema#string>

Figure 2.20: SPARQL Query 4 results.

Find Meals Containing a Given Allergen

```
1 PREFIX : <http://menu.ch#>
2
3 SELECT DISTINCT ?MealName
4 WHERE {
5     VALUES ?allergen { :Fish }
6
7     ?meal :hasIngredient ?ingredient ;
8           :name ?MealName .
9
10    ?ingredient :containsAllergen ?allergen .
11 }
12 ORDER BY ?MealName
```

Code 2.18: SPARQL Query 5

This query identifies meals that contain a specified allergen (Fish) by examining whether any ingredient in a meal carries that allergen. The results are shown in the Figure 2.21 below.

MealName
"Salmone in Crosta" ^{^^} <http://www.w3.org/2001/XMLSchema#string>
"Spaghetti alle vongole" ^{^^} <http://www.w3.org/2001/XMLSchema#string>

Figure 2.21: SPARQL Query 5 results.

Find Ingredients in a Given Meal

```

1 PREFIX : <http://menu.ch#>
2
3 SELECT DISTINCT ?IngredientName
4 WHERE {
5     VALUES ?meal { :Tiramisu }
6
7     ?meal :hasIngredient ?ingredient .
8     ?ingredient :name ?IngredientName .
9 }
10 ORDER BY ?IngredientName

```

Code 2.19: SPARQL Query 6

This query retrieves the names of all ingredients in a specific meal (Tiramisu). Useful for checking the composition of the meal. The results are shown in the Figure 2.22 below.

IngredientName
"Cocoa Powder" ^{^^} <http://www.w3.org/2001/XMLSchema#string>
"Coffee" ^{^^} <http://www.w3.org/2001/XMLSchema#string>
"Eggs" ^{^^} <http://www.w3.org/2001/XMLSchema#string>
"Lady Fingers" ^{^^} <http://www.w3.org/2001/XMLSchema#string>
"Mascarpone Cheese" ^{^^} <http://www.w3.org/2001/XMLSchema#string>
"Sugar" ^{^^} <http://www.w3.org/2001/XMLSchema#string>

Figure 2.22: SPARQL Query 6 results.

Find Meals a Guest Can Eat using SWRLS rules

It is also possible to query using the new inferred properties. This is a simplified version of the first query, which eliminates the need to directly check ingredient allergens by simply filtering out dishes related to the guest via the `notSuitableFor` property. This reduces query complexity and leverages reasoning output. The result of the simplified query is the same as the original as shown in Figure 2.17

```

1 PREFIX : <http://menu.ch#>
2

```

```

3 SELECT ?Meal (SUM(?cal) AS ?Calories)
4 WHERE {
5   VALUES ?guest { :Guest2 }
6
7   ?guest :hasDietType ?diet ;
8           :gMaxCalories ?maxCalories .
9
10  ?diet :canEat ?dish .
11  ?dish :name ?Meal ;
12         :hasIngredient ?ingredient .
13
14  ?ingredient :hasCalories ?cal .
15
16  FILTER NOT EXISTS {
17    ?dish :notSuitableFor ?guest .
18  }
19 }
20 GROUP BY ?Meal ?maxCalories
21 HAVING (SUM(?cal) <= ?maxCalories)

```

Code 2.20: SPARQL Query 1 using SWRL Rules

2.3.6 SHACL Shapes

SHACL (Shapes Constraint Language) is used to validate RDF data against a set of constraints. SHACL shapes are defined to ensure the integrity and completeness of information in the food menu ontology. Each shape targets a specific class (like Guest, Ingredient, Dish, etc.) and specifies required properties, data types, and cardinality rules to validate instances of that class.

```

1 :GuestShape
2   a sh:NodeShape ;
3   sh:targetClass :Guest ;
4
5   sh:property [
6     sh:path :name ;
7     sh:datatype xsd:string ;
8     sh:minCount 1 ;
9     sh:maxCount 1 ;
10    sh:message "A guest must have a name." ;
11  ] ;
12
13  sh:property [
14    sh:path :hasDietType ;
15    sh:minCount 1 ;
16    sh:maxCount 1 ;
17    sh:class :Diet ;
18    sh:message "A guest must have exactly one diet type." ;
19  ] ;
20
21  sh:property [
22    sh:path :gMaxCalories ;
23    sh:datatype xsd:int ;
24    sh:minCount 1 ;
25    sh:maxCount 1 ;

```

```

26     sh:message "A guest must have a maximum calorie limit specified
      as an integer." ;
27 ] ;
28
29 sh:property [
30     sh:path :hasAllergy ;
31     sh:minCount 0 ;
32     sh:class :Allergen ;
33     sh:message "A guest may have zero or more allergies." ;
34 ] .

```

Code 2.21: Guest Shape

The *GuestShape* applies to all individuals of type *Guest*. It ensures that a guest must have a name specified as a string and must be associated with exactly one diet type from the *Diet* class. Additionally, each guest must define a maximum calorie limit as an integer. The shape also supports optional associations to allergies, allowing guests to have zero or more linked allergens from the *Allergen* class.

```

1 :IngredientShape a sh:NodeShape ;
2   sh:targetClass :Ingredient ;
3   sh:property [
4     sh:path :name ;
5     sh:datatype xsd:string ;
6     sh:minCount 1 ;
7     sh:message "An ingredient must have a name." ;
8   ] ;
9   sh:property [
10    sh:path :hasCalories ;
11    sh:datatype xsd:int ;
12    sh:minCount 1 ;
13    sh:maxCount 1 ;
14    sh:message "An ingredient must have calories specified as an
      integer." ;
15  ] ;
16  sh:property [
17    sh:path :containsAllergen ;
18    sh:minCount 0 ;
19    sh:class :Allergen ;
20    sh:message "An ingredient may have zero or more allergens." ;
21  ] ;
22  sh:property [
23    sh:path :fitForDiet ;
24    sh:class :Diet ;
25    sh:minCount 1 ;
26    sh:message "An ingredient can be suitable for one or more diets
      ." ;
27  ] .

```

Code 2.22: Ingredient Shape

The *IngredientShape* validates individuals of type *Ingredient*. Each ingredient must have a name (string) and a calorie value (int), with exactly one calorie value required. It may optionally be associated with one or more allergens from the *Allergen* class. Furthermore, every ingredient must be compatible with at least one diet, enforced via the *fitForDiet* property pointing to the *Diet* class.


```

1 :AllergenShape a sh:NodeShape ;
2   sh:targetClass :Allergen ;
3   sh:property [
4     sh:path :name ;
5     sh:datatype xsd:string ;
6     sh:minCount 1 ;
7     sh:maxCount 1 ;
8     sh:message "An allergen must have a name." ;
9   ] .

```

Code 2.23: Allergen Shape

The *AllergenShape* applies to individuals of type Allergen. It enforces a single required property: each allergen must have a name specified as a string. The name must occur exactly once.

```

1 :DietShape a sh:NodeShape ;
2   sh:targetClass :Diet ;
3   sh:property [
4     sh:path :name ;
5     sh:datatype xsd:string ;
6     sh:minCount 1 ;
7     sh:maxCount 1 ;
8     sh:message "A diet must have a name." ;
9   ] ;
10  sh:property [
11    sh:path :canEat ;
12    sh:class :Dish ;
13    sh:minCount 1 ;
14    sh:message "A diet must list at least one dish it can eat." ;
15  ] .

```

Code 2.24: Diet Shape

The *DietShape* is used to validate individuals of type Diet. Each diet must have a name (string) and must list at least one dish it permits, using the canEat property linked to the Dish class. This ensures that diets are defined with meaningful and applicable meal options.

```

1 :DishShape
2   a sh:NodeShape ;
3   sh:targetClass :Dish ;
4
5   # Dish must have a name (string)
6   sh:property [
7     sh:path :name ;
8     sh:datatype xsd:string ;
9     sh:minCount 1 ;
10    sh:maxCount 1 ;
11    sh:message "A dish must have a name." ;
12  ] ;
13
14  sh:property [
15    sh:path :hasIngredient ;
16    sh:minCount 1 ;
17    sh:class :Ingredient ;

```

```
18   sh:message "A dish must have at least one ingredient." ;  
19 ] .
```

Code 2.25: Dish Shape

The *DishShape* ensures that all individuals of type *Dish* include necessary properties. Every dish must have a name (string) and must include at least one ingredient, linked via the *hasIngredient* property to individuals of type *Ingredient*. This guarantees that no dish is undefined or empty.

These SHACL shapes collectively define the structural rules that all data in the ontology must follow. They play a critical role in data validation, helping to catch modeling errors early and ensuring consistency across the knowledge base.

3. Agile and Ontology-based Meta-modelling

This chapter introduces our Agile Meta-Modeling approach and demonstrates how we customized a Business Process Model and Notation (BPMN) 2.0 process. First, we used AOAME, a tool for creating flexible knowledge graph schemas, to extend the standard BPMN notation and create custom classes with specific attributes. Next, we adapted a SPARQL query to dynamically retrieve values from the BPMN instead of using fixed values in the query. We then tested the queries using Jena Fuseki.

3.1 BPMN 2.0 Model

Figure 3.1 illustrates the BPMN 2.0 process for generating a personalized menu for a client. The diagram is organized into a pool with two lanes: the top lane represents the client’s interactions, while the bottom lane corresponds to the server-side or system processes.

The workflow begins when the client scans a QR code. At this point, the client can either use previously saved information (for example, by logging in or saved cookies, depending on the implementation on the client side) or manually enter their preferences, including diet type, maximum calorie count, and a list of allergens. To process this information, we extended the standard BPMN Task class to create our own specialized class called *suggestMeal*. This new class is then used twice in the diagram to represent the two processing paths. The first instance, labeled *Suggested Meals By Guest Profile*, is for users that sent their previously ”saved” profile, while the second, *Suggested Meals By Details*, handles manually entered preferences. Both tasks result in a selection of recommended meals, which are then displayed to the client. The process concludes when the user selects a meal, terminating the workflow.

The design of our ontology led us to a query for meal suggestions based on a single input because it includes saved user profiles. However, we also wanted to explore a multiple-input approach, which is why we divided the workflow into ”saved” and ”new” users.

As detailed in Figure 3.2, the two tasks require different input attributes. The image on the right shows that the *Suggested Meals By Guest Profile* task takes a single attribute: the client’s name. The image on the left illustrates that *Suggested Meals By Details* requires three attributes: *dietType*, *maxCalories*, and a list of *allergies*.

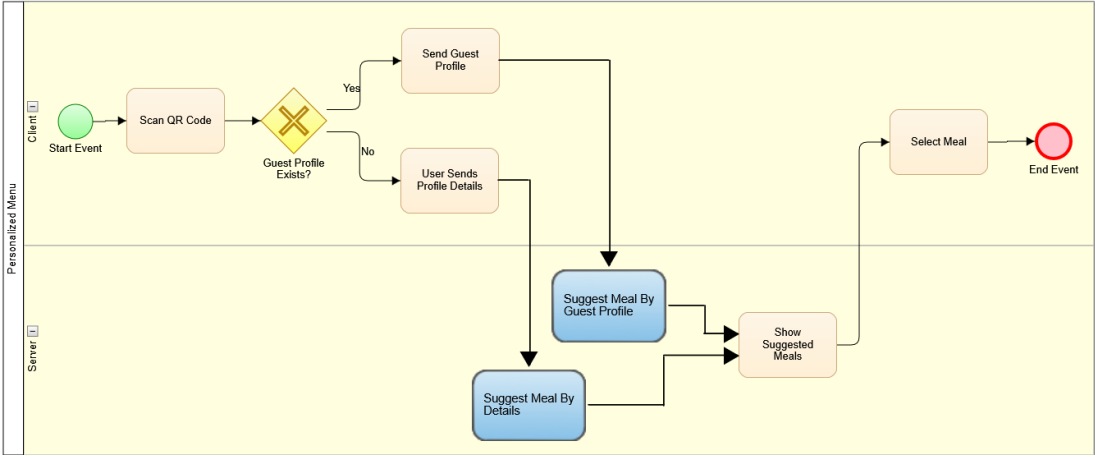


Figure 3.1: BPMN 2.0 Model using AOAME

Model element attributes

ID: suggestMeal_cb141a48-eeb2-42b9-a047-275b79d4611c
Instantiation Type: Instance

Relation	Value	Actions
dietType	Vegetarian	<button>Remove</button>
maxCalories	3000	<button>Remove</button>
allergies	Gluten,Lactose	<button>Remove</button>

▼ Add Relation

Save Close

Model element attributes

ID: suggestMeal_4ff4dc2a-8de0-4d60-843f-50e64e55f8a7
Instantiation Type: Instance

Relation	Value	Actions
guestName	Guest2	<button>Remove</button>

▼ Add Relation

Save Close

Figure 3.2: Custom tasks properties

3.2 Jena Fuseki Queries

In this section, we will demonstrate the query executed using Jena Fuseki. The function is similar to the one described in Section 2.3.5, but with a key difference: preferences are retrieved directly from the extended tasks in the BPMN, rather than being defined within the query.

3.2.1 Prefixes

```

1 PREFIX io: <https://iaco.me/>
2 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
3 PREFIX owl: <http://www.w3.org/2002/07/owl#>
4 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
5 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
6 PREFIX mod: <http://fhnw.ch/modelingEnvironment/ModelOntology#>
7 PREFIX lo: <http://fhnw.ch/modelingEnvironment/LanguageOntology#>
8 PREFIX : <http://menu.ch#>

```

Code 3.1: Prefixes used in Jena Fuseki

3.2.2 First query

This query identifies all meals that a specific guest can consume, based on their type of diet, allergy restrictions, and maximum allowed calories. It begins by retrieving the guest profile name from the first instance of the BPMN class *suggestMeal*, using its unique identifier. Then, a BIND clause, which allows a value to be assigned to a new variable from an expression, is used to construct a unique IRI for the guest: *BIND(IRI(CONCAT(STR(:), ?guest)) AS ?guestIRI)*. This is essential because the guest's name, initially a base IRI, cannot be directly used to navigate the graph and retrieve related data. By converting the default namespace (*STR(:)*) and the guest's name (*?guest*) into a single string (*CONCAT*) and then to a proper IRI (*IRI*), we create a valid absolute identifier. This *?guestIRI* is subsequently used to retrieve the guest's diet and calorie limit. The query then matches the dishes permitted by that diet and sums up the calories of their ingredients. *FILTER NOT EXISTS* block ensures that no ingredient in a potential dish contains allergens to which the guest is allergic. The result is then grouped by meal name and further filtered by the total calorie constraint using the *HAVING* clause, making sure only suitable options are presented.

```

1 SELECT ?Meal (SUM(?cal) AS ?Calories)
2 WHERE {
3   mod:suggestMeal_4ff4dc2a-8de0-4d60-843f-50e64e55f8a7
4     lo:guestName ?guest .
5
6   BIND( IRI(CONCAT(STR(:), ?guest)) AS ?guestIRI ) .
7
8   ?guestIRI :hasDietType ?diet ;
9             :gMaxCalories ?maxCalories .
10
11  OPTIONAL { ?guestIRI :hasAllergy ?guestAllergy . }
12
13  ?diet :canEat ?dish .
14  ?dish :name ?Meal ;

```

```

15      :hasIngredient ?ingredient .
16
17      ?ingredient :hasCalories ?cal .
18
19      FILTER NOT EXISTS {
20          ?guestIRI :hasAllergy ?guestAllergy .
21          ?dish :hasIngredient ?dishIngredient .
22          ?dishIngredient :containsAllergen ?guestAllergy .
23      }
24
25  }
26  GROUP BY ?Meal ?maxCalories
27  HAVING (SUM(?cal) <= ?maxCalories)

```

Code 3.2: First query

The output of the query is shown in the figure 3.3 below.

	Meal	Calories
1	Spaghetti alle vongole	"941"^^<http://www.w3.org/2001/XMLSchema#integer>
2	Tiramisù	"1583"^^<http://www.w3.org/2001/XMLSchema#integer>
3	Scaloppine al limone	"1816"^^<http://www.w3.org/2001/XMLSchema#integer>
4	Salmone in Crosta	"1038"^^<http://www.w3.org/2001/XMLSchema#integer>
5	Sorbetto al limone	"416"^^<http://www.w3.org/2001/XMLSchema#integer>
6	Lasagna	"745"^^<http://www.w3.org/2001/XMLSchema#integer>
7	Misto Verdure	"508"^^<http://www.w3.org/2001/XMLSchema#integer>
8	Guanciale Croccante con Uova	"1734"^^<http://www.w3.org/2001/XMLSchema#integer>
9	Pasta al Burro, Parmigiano e Pepe Nero	"1753"^^<http://www.w3.org/2001/XMLSchema#integer>
10	Bistecca alla Fiorentina	"505"^^<http://www.w3.org/2001/XMLSchema#integer>
11	Omelette al Parmigiano con Cipolle e Prezzemolo	"1622"^^<http://www.w3.org/2001/XMLSchema#integer>
12	Carbonara	"1790"^^<http://www.w3.org/2001/XMLSchema#integer>
13	Spaghetti al pomodoro	"789"^^<http://www.w3.org/2001/XMLSchema#integer>
14	Crema Caffè	"1052"^^<http://www.w3.org/2001/XMLSchema#integer>
15	Involtini di Manzo al Parmigiano	"1371"^^<http://www.w3.org/2001/XMLSchema#integer>
16	Riso con Pomodoro, Cipolla e Prezzemolo	"729"^^<http://www.w3.org/2001/XMLSchema#integer>
17	Risotto alla milanese	"1628"^^<http://www.w3.org/2001/XMLSchema#integer>

Figure 3.3: Jena Fuseki query 1 using dynamic data from BPMN 2.0.

3.2.3 Second query

This query identifies meals a guest can consume based on their diet, calorie limit, and allergies, similar to the previous one. However, instead of retrieving a guest's profile, it directly receives the guest's diet type, maximum calories, and a string of allergies from

the second instance of the BPMN class suggestMeal. It then uses multiple BIND clauses with IF, CONTAINS, and LCASE() to parse this allergy string into specific allergen IRIs (e.g., for "gluten," "lactose," etc.). LCASE() ensures case-insensitive matching, while CONTAINS() checks for the presence of specific allergy keywords within the string. The FILTER NOT EXISTS block then leverages these specific allergen IRIs to strictly exclude any dishes containing those allergens. The query concludes by adding the calories of the ingredients and filtering by the maximum calorie limit using HAVING.

```

1 SELECT ?Meal (SUM(?cal) AS ?Calories)
2 WHERE {
3   mod:suggestMeal_cb141a48-eeb2-42b9-a047-275b79d4611c
4     lo:dietType ?diet ;
5     lo:maxCalories ?maxCalories;
6     lo:allergies ?guestAllergy .
7
8   BIND( IRI(CONCAT(STR(:), ?diet)) AS ?dietIRI)
9
10  BIND(
11    IF(CONTAINS(LCASE(?guestAllergy), "gluten"),
12      IRI(CONCAT(STR(:), "Gluten")),
13      IRI(CONCAT(STR(:), ""))
14    ) AS ?guestGlutenAllergyIRI
15  )
16
17  BIND(
18    IF(CONTAINS(LCASE(?guestAllergy), "lactose"),
19      IRI(CONCAT(STR(:), "Lactose")),
20      IRI(CONCAT(STR(:), ""))
21    ) AS ?guestLactoseAllergyIRI
22  )
23
24  BIND(
25    IF(CONTAINS(LCASE(?guestAllergy), "fish"),
26      IRI(CONCAT(STR(:), "Fish")),
27      IRI(CONCAT(STR(:), ""))
28    ) AS ?guestFishAllergyIRI
29  )
30
31  BIND(
32    IF(CONTAINS(LCASE(?guestAllergy), "eggsallergen"),
33      IRI(CONCAT(STR(:), "EggsAllergen")),
34      IRI(CONCAT(STR(:), ""))
35    ) AS ?guestEggsAllergyIRI
36  )
37
38  BIND(
39    IF(CONTAINS(LCASE(?guestAllergy), "nuts"),
40      IRI(CONCAT(STR(:), "Nuts")),
41      IRI(CONCAT(STR(:), ""))
42    ) AS ?guestNutsAllergyIRI
43  )
44
45  ?dietIRI :canEat ?dish .
46  ?dish :name ?Meal ;

```

```

47      :hasIngredient ?ingredient .
48
49      ?ingredient :hasCalories ?cal .
50
51      FILTER NOT EXISTS {
52          ?dish :hasIngredient ?dishIngredient .
53          ?dishIngredient :containsAllergen ?allergenFromDish .
54
55          FILTER (
56              (?allergenFromDish = ?guestGlutenAllergyIRI && ?
57                  guestGlutenAllergyIRI != IRI(CONCAT(STR(:), "")))
58              || (?allergenFromDish = ?guestLactoseAllergyIRI && ?
59                  guestLactoseAllergyIRI != IRI(CONCAT(STR(:), "")))
60              || (?allergenFromDish = ?guestFishAllergyIRI && ?
61                  guestFishAllergyIRI != IRI(CONCAT(STR(:), "")))
62              || (?allergenFromDish = ?guestEggsAllergyIRI && ?
63                  guestEggsAllergyIRI != IRI(CONCAT(STR(:), "")))
64              || (?allergenFromDish = ?guestNutsAllergyIRI && ?
65                  guestNutsAllergyIRI != IRI(CONCAT(STR(:), "")))
66          )
67      }
68  }
69
70  GROUP BY ?Meal ?maxCalories
71  HAVING (SUM(?cal) <= ?maxCalories)

```

Code 3.3: Second query

The output of the query is shown in the figure 3.4 below.

	Meal	Calories
1	Sorbetto al limone	"416"^^<http://www.w3.org/2001/XMLSchema#integer>
2	Misto Verdure	"508"^^<http://www.w3.org/2001/XMLSchema#integer>

Figure 3.4: Jena Fuseki query 2 using dynamic data from BPMN 2.0.

4. Conclusions

4.1 Leonardo Salpini

I will start by discussing Trisotech and decision tables. I found the tool to be user-friendly at the most basic level, but it becomes more complicated when FEEL expressions and other complex elements are introduced. Initially, I had difficulty understanding how to structure the tables because we were focusing too much on FEEL expressions, iterators, and other complex elements of the tool. For instance, we tried to calculate the total calories of a dish by adding up the calories of each ingredient. Since these elements weren't covered in class, my colleague and I decided to simplify the tables by using the total calories for each plate instead of summing each ingredient's value. In the final decision, we only used a list contains FEEL expression, but I think it was necessary to make our approach work. It might be possible to do without it, but nothing came to our mind.

Next, regarding Prolog, I found this to be the most enjoyable part of the project. We had some issues at the beginning while trying to use recursion, but the development of the program was pretty straightforward after that. Here, we were also able to calculate the total calories of a dish by summing the calories of all its ingredients, which we could not be able to do using decision tables. I'm pretty happy with the result.

Working with Protégé and GraphDB was straightforward. We primarily edited the .ttl file directly and used GraphDB to visualize the ontology graph. We used Protégé to run SPARQL queries, validate SHACL shapes, and apply SWRL rules for more advanced tasks. The only issue we encountered was duplicated triples, which caused some queries to run slowly. We weren't sure what caused it, but restarting the program solved the problem. Overall, I enjoyed this part of the project. It provided a great introduction to how ontology-based systems can meaningfully represent knowledge and how reasoning can reveal new connections within data.

Finally, I will conclude by discussing AOAME and Jena Fuseki. I found this to be the most annoying part of the project, though not because of the arguments themselves, which were interesting. However, the large quantity of problems we encountered completely overshadowed the good parts. We had some issues during installation and experienced numerous bugs and crashes while using the tool. This forced us to restart Docker many times. Due to these issues, I didn't enjoy this part of the project at all.

4.2 Mohit Vijay Saini

The first part of the project seemed relatively simple. I had some issues with the way decision tables work, but after some experimentation, I learned the basics pretty quickly. Nevertheless, it still proved challenging to structure our menu into multiple

sub-decisions. I had to resort to FEEL expressions and lists as this was the only way, I could imagine working with decision tables because it felt closer to programming. Its probably possible to avoid FEEL but in the end, we both decided to use them as they felt necessary. Decision tables offer a clear structure, but i can see them becoming difficult to manage when handling multiple sub-decisions.

Working with Prolog for the first time was interesting. Because of its declarative language, I got to think differently about solving problems, focusing on describing facts and rules instead of writing step-by-step instructions. Prolog's way of searching for solutions made the process feel straightforward but debugging or scaling the rule base was demanding. Overall, Prolog gave a different perspective on programming and showed how powerful logic-based reasoning can be for handling complex conditions and choices.

Protégé served as a powerful platform for organizing and managing information by facilitating the creation of ontologies. Primarily, I worked directly on the .ttl file and imported it to Protégé or GraphDB. Protégé was used more due to its support for not only SPARQL queries but also SHACL validation and SWRL rules. GraphDB on the other hand, offered a much better visual representation of the ontology graph. Both tools were straightforward to use, and my existing familiarity with SQL eased the learning curve for SPARQL. However, I encountered an unusual issue in Protégé where the triples were duplicated, resulting in repeated outputs during query execution. Despite this, this part of the project was very interesting as I had always used SQL type databases to store data, but this hands-on experience highlighted the power of ontology-based systems and how reasoning and inference can be used to uncover implicit relationships within the data.

For the final part of the project, AOAME was challenging to work with. Installing the tool using docker was, for the most part, not the biggest issue. There was an error during the setup phase, but ignoring it had no consequences. The biggest issue for me were the bugs encountered during the modeling phase. There were instances where selecting a dotted arrow would still result in a solid arrow. Sometimes, connecting two activities together with a connector would result in the arrow becoming a big black box. The solution that worked for me was to completely wipe the docker images and re-install them. Upon opening a saved model, all activities, connectors, and events would move, resulting in a messy diagram. The last issue I had was the auto saving, as I noticed that if I did anything while the tool was saving, it would get stuck saving forever. To avoid this, I had to stop after every change and wait for the save to complete before another change. On the other hand, I had no issues with Jena-Fuseki triple store as I was able to import our ontology and run modified SPARQL queries using input from the model without any problems. Overall, This part of the project was frustrating and i believe that the tool should be further improved by focusing on fixing bugs and adding some QOL updates, particularly for the user interface.